

EXAMINATION ASSIGNMENT

Universal AI API Connector & API Hub

Technical Development Evaluation

1. Assignment Objective

Build a complete, production-style web application that acts as a central AI API Hub. The system must allow an administrator to create reusable AI-powered API endpoints by defining what an API does, what inputs it accepts, which AI provider and model it uses, what instructions should be sent to the model, and what structured response should be returned.

The final system must be live, hosted online, responsive on desktop and mobile, and stable enough to be tested directly through a browser and through API requests.

2. Core Problem to Solve

The goal is to avoid building separate AI integrations for every new use case. Instead, build one platform from which new AI APIs can be configured and generated.

Example:

- 🕒 Card Scanner: image input → AI processing → structured contact JSON.
- 🕒 Article Writer: topic and writing parameters → AI processing → structured article JSON.
- 🕒 Invoice Scanner: invoice image → AI processing → structured invoice JSON.
- 🕒 Content Rewriter: text and instructions → AI processing → structured rewritten content.

3. How the System Should Work

1. Administrator opens the API Hub.
2. Administrator creates a new API and gives it a name and description.
3. Administrator selects an AI provider and model.
4. Administrator defines the instructions/prompt for that API.
5. Administrator defines the input fields and validation rules.
6. Administrator defines the expected output structure/schema.
7. The system saves the configuration and generates a unique API endpoint.
8. The system automatically generates documentation for the new endpoint.
9. An external application sends a POST request to the generated endpoint.
10. The connector validates the request, sends the required data to the selected AI provider, processes the response, validates the output, records usage statistics, and returns structured data.

4. API Creation Interface

The web interface must allow an administrator to create and manage connectors. At minimum, provide:

- 🕒 API Name
- 🕒 Description / Purpose
- 🕒 AI Provider
- 🕒 AI Model
- 🕒 System Instructions / Prompt

- ⌚ Input Parameters
- ⌚ Output Schema
- ⌚ API access/authentication configuration
- ⌚ Active / Disabled status

5. Multiple AI Providers

Multiple providers are mandatory. Students must integrate more than one AI provider using their own available free/developer API access.

The application should be designed around a provider-adapter approach so that adding another provider does not require rewriting the entire system.

The system should support provider/model configuration rather than being hardcoded to one AI service.

6. Model Selection

Basic implementation:

- ⌚ Allow the administrator to manually enter/select the model identifier.

Advanced implementation:

- ⌚ Provide a model dropdown for supported providers.
- ⌚ Provide an Update/Refresh Models action that fetches the latest available models from the provider when the provider supports model listing.
- ⌚ Store and display the currently available model information.

The model-management system is an enhancement. The core assignment is the working multi-provider connector architecture.

7. Dynamic Input System

Different connectors must be able to accept different types of input. The administrator should be able to define fields such as:

Field	Type	Required	Example
image	Image/File	Yes	Business card
topic	Text	Yes	Digital marketing
keywords	Text	Optional	SEO, AI, automation
word_count	Number	Optional	1200
options	JSON	Optional	{"tone":"professional"}

Support, at minimum:

- ⌚ Text
- ⌚ Number
- ⌚ Boolean
- ⌚ Image
- ⌚ File
- ⌚ JSON

Each parameter should support a name, type, required/optional status, description, and where appropriate, validation/default values.

8. Prompt / Instruction Engine

Every connector must have its own instructions. The connector should combine the configured instructions with the submitted user input before calling the selected AI provider.

Example instruction for a Card Scanner:

You are a business card extraction system.

Extract the person's name, company, designation, phone, email and website from the supplied image.

Return only valid JSON matching the configured output structure.

9. Structured Output

The administrator must be able to define an expected output schema.

```
{
  "name": "string",
  "company": "string",
  "designation": "string",
  "phone": "string",
  "email": "string",
  "website": "string"
}
```

The API should validate or normalize the AI response where possible and return a predictable response format.

```
{
  "success": true,
  "data": {},
  "error": null
}
```

10. Automatically Generated API Documentation

Whenever a connector is created, the platform must generate documentation for that connector.

- 🕒 Endpoint URL and HTTP method
- 🕒 Purpose and description
- 🕒 Authentication requirements
- 🕒 Request format
- 🕒 Required and optional parameters
- 🕒 Input types
- 🕒 Example request
- 🕒 Expected response
- 🕒 Error response formats
- 🕒 Example cURL request
- 🕒 Provider and model information where appropriate

The documentation should be usable by another developer without needing to inspect the source code.

11. API Testing Interface

Every connector should have a Test API function in the web interface.

- 🕒 Display required input fields dynamically.

- ⌚ Allow the administrator to submit a test request.
- ⌚ Display request/processing status.
- ⌚ Display the returned response.
- ⌚ Display errors clearly.
- ⌚ Display response time and available usage information.

12. Usage Statistics and Logging

Usage statistics are a core requirement of this assignment. Every API request must be logged and stored so that the administrator can understand how the connector is performing.

- ⌚ Total requests
- ⌚ Successful requests
- ⌚ Failed requests
- ⌚ Last used
- ⌚ First used
- ⌚ Response time
- ⌚ Input tokens, where available
- ⌚ Output tokens, where available
- ⌚ Total tokens, where available
- ⌚ Estimated cost, where the provider exposes enough information to calculate it
- ⌚ Provider
- ⌚ Model
- ⌚ Error type/message
- ⌚ Request timestamp

Statistics should be stored persistently in a database or equivalent persistent storage. They must not disappear after a page refresh or server restart.

13. API Management Dashboard

Create a central dashboard showing all configured connectors.

API	Provider	Model	Input	Status	Requests	Last Used
Card Scanner	Provider A	Vision Model	Image	Active	1,248	Today
Article Writer	Provider B	Text Model	Text	Active	421	Today

The administrator should be able to create, edit, disable, delete, test, document, and inspect statistics for each connector.

14. Generated API Endpoint

A created connector must expose a real HTTP API endpoint.

Example:

POST /api/card-scanner

For an image-based connector, an external application should be able to send an image to the endpoint and receive the configured structured response.

15. Security and API Key Handling

- ⌚ AI provider API keys must remain on the server/backend.

- ⌚ Provider keys must not be exposed in frontend JavaScript, public documentation, screenshots, or API responses.
- ⌚ Students must use their own free/developer API credentials for evaluation.
- ⌚ Do not share your API credentials with other students or publish them publicly.
- ⌚ Generated endpoints should have an appropriate access mechanism such as an application API key.
- ⌚ Validate incoming requests and restrict invalid or oversized input where appropriate.

16. Live Hosting Requirement

The completed system must be hosted online. The evaluator must be able to open the web application and test it without creating an account or logging into the application.

- ⌚ Localhost-only submissions will not be accepted.
- ⌚ Students may use a suitable free hosting provider or other hosting environment.
- ⌚ The application must remain accessible during evaluation.
- ⌚ The deployed application must not depend on the student's personal computer being switched on.
- ⌚ The final submission must include the live application URL, working API endpoint(s), and API documentation URL.

17. Technology

The preferred implementation is PHP, but students may use another suitable technology if they can deliver the required functionality.

- ⌚ The entire system must be built from scratch.
- ⌚ Students may use AI tools, libraries, frameworks, and documentation as development aids.
- ⌚ The final implementation remains the student's responsibility.
- ⌚ The system must be responsive and usable on both PC and mobile.
- ⌚ The focus is on reliability, architecture, maintainability, security, and working functionality.

18. Mandatory Demonstration

At least two working connectors must be demonstrated, using multiple AI providers.

The demonstrations must cover different types of processing. For example:

- ⌚ Connector A: image input → AI vision processing → structured JSON.
- ⌚ Connector B: text input → AI text processing → structured JSON.

Students may choose other use cases if they demonstrate the flexibility of the platform.

19. Bonus Features

The following are optional enhancements. They are not required for the core assignment, but can demonstrate deeper understanding and engineering effort.

- ⌚ Automatic model discovery and model dropdown updates.
- ⌚ Support for an image-generation API where a prompt produces an image response.
- ⌚ Handling image outputs as files, URLs, or Base64 depending on the provider.
- ⌚ Saving generated files/images and associating them with API request logs.
- ⌚ More AI providers.
- ⌚ Retry and fallback provider/model logic.
- ⌚ Rate limiting and request quotas.

- ⌚ Advanced analytics and charts.
- ⌚ API versioning.
- ⌚ Webhook/callback support.
- ⌚ API playground with request/response history.
- ⌚ Import/export of connector configurations.
- ⌚ Automatic schema validation and response repair.

20. Reliability Expectations

The system should be engineered as a robust application, not as a simple proof of concept. It should handle invalid input, provider failures, malformed AI responses, timeouts, unavailable models, and unexpected API errors gracefully.

- ⌚ The application should not expose raw server errors to users.
- ⌚ Failures should be logged with enough information to diagnose the problem.
- ⌚ A failed AI request should be recorded as a failed request.
- ⌚ The system should handle provider/API timeouts appropriately.
- ⌚ Database operations and API logging should not cause the entire application to fail.
- ⌚ The interface should provide clear status and error feedback.

21. Final Submission

Submit the following:

- ⌚ Live web application URL
- ⌚ Working API endpoint(s)
- ⌚ API documentation URL/page
- ⌚ At least two configured and working connectors
- ⌚ Details of the AI providers used
- ⌚ Sample test inputs
- ⌚ Sample API responses
- ⌚ Brief architecture explanation
- ⌚ Hosting/deployment details
- ⌚ Any bonus features implemented

Source code may be reviewed if required, but submitting source code alone does not satisfy the assignment. The live hosted system is the primary deliverable.

22. Evaluation Principle

The system will be evaluated as a real technical product. A working, stable, secure and well-designed API Hub is more important than a visually attractive prototype. The application should demonstrate that the student understands API integration, backend architecture, AI provider abstraction, dynamic configuration, structured data handling, logging, statistics, deployment, and error handling.